

ECE 6256 Assignment 1A: ATIS Slot Filling — CRF vs. BiLSTM

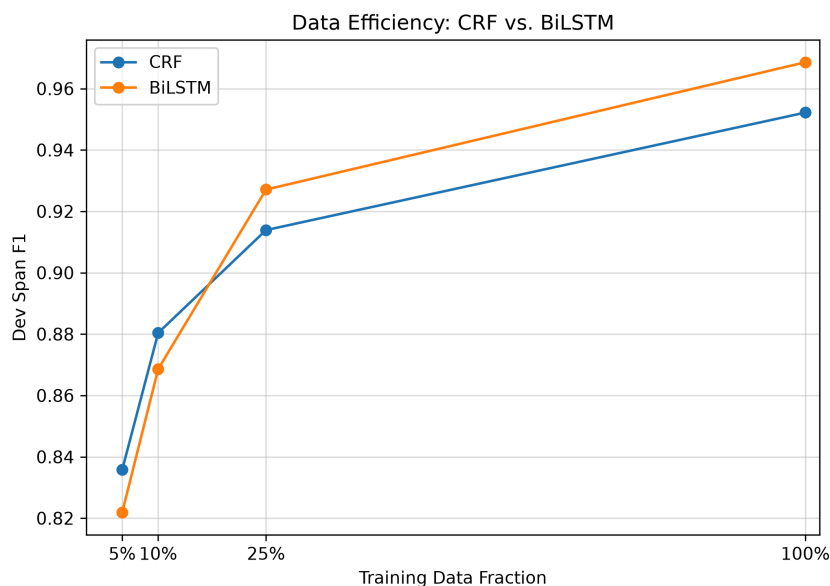


Figure 1. Data-efficiency curve using dev span F1. CRF and BiLSTM were trained on the same fixed 5%, 10%, 25%, and 100% training subsets.

Environment

Experiments were performed using the `CAI_S0_FUN` conda environment with Python 3.12.14 and a CUDA-enabled PyTorch 2.14.0 installation. BiLSTM training was run on an NVIDIA H100 GPU because CPU training was substantially slower. CRF training was performed without requiring GPU acceleration.

Part A — CRF: 2009 Experience

The CRF was implemented using `sklearn-crfsuite`, with the main emphasis on hand-engineered features. Starting with only lowercase word identity produced a dev span F1 of 0.7731. The largest improvement came from adding the previous and next words together with BOS and EOS indicators. This increased dev span F1 to 0.9320.

Experiment	Features	Dev Span F1
1	Word identity	0.7731

Experiment	Features	Dev Span F1
2	+ casing / shape	0.7713
3	+ affixes	0.7820
4	+ previous / next word + BOS / EOS	0.9320
5	+ ± 2 context window	0.9320

The final CRF configuration included word identity, casing, digit features, affixes, a ± 2 context window, BOS/EOS information, and neighboring-word bigram features. Retraining this final configuration on 100% of the training set produced a **dev span F1 of 0.9522**. On the test set, CRF obtained: **Token F1: 0.9287, Span F1: 0.9310**.

Part B — BiLSTM: 2014 Experience

The BiLSTM uses a **100-dimensional trainable embedding layer**, followed by a bidirectional LSTM and a linear projection into the slot-label space. Sequences are packed before being passed through the LSTM so that padding does not affect either direction of the network. The final 100% training run achieved: **Dev span F1: 0.9686, Test token F1: 0.9475, Test span F1: 0.9472**. The final BiLSTM therefore outperformed the final CRF on both the development and test sets. A handwritten Viterbi decoder was also implemented and explored as the optional bonus component.

Part C — Data Efficiency

Both models were trained on the required fixed **5%, 10%, 25%, and 100%** subsets of the ATIS training set. The same subsamples were used for both models, while the random seed controlled model-training randomness. The reported values are **development-set span F1**.

Training Fraction	CRF Dev F1	BiLSTM Dev F1	Difference
5%	0.8358	0.8219	CRF +0.0139
10%	0.8804	0.8686	CRF +0.0118
25%	0.9139	0.9271	BiLSTM +0.0132
100%	0.9522	0.9686	BiLSTM +0.0164

The CRF performs better when only a small amount of labeled data is available. At 5% and 10%, its hand-designed lexical and contextual features provide useful prior structure without requiring the model to learn all those patterns from the training examples. However, the advantage is now small: 1.39 F1 points at 5% and 1.18 points at 10%. As training data increases, the BiLSTM improves more rapidly. At 25%, it reaches 0.9271 and overtakes the CRF's 0.9139. At 100%, the BiLSTM reaches 0.9686 compared with 0.9522 for the CRF. The experimentally observed crossover occurs **between 10% and 25% of the training set**.

Part D — Error Analysis

Ten test-set failures were examined: four CRF-only errors, three BiLSTM-only errors, and three errors made by both models. Full examples are provided in `Error_Analysis.txt`. The CRF's main weakness is **arrival/departure role confusion**. It often identifies the correct city, date, or time span but assigns the wrong semantic role, especially when cues such as “arrive” are separated from the slot. This suggests that its local features are strong for span detection but weaker for longer-range semantic relationships. The BiLSTM handles broader context better but can **overgeneralize contextual cues**. For example, an earlier “arrive” cue can incorrectly influence later date or time expressions. It also occasionally confuses closely related labels, such as different return-date categories. Both models struggle with some **ground-transportation queries**, where generic `airport_name` or `city_name` labels are expected but the models instead predict flight-oriented `fromloc` or `to loc` labels. Overall, the CRF is stronger at local pattern recognition, while the BiLSTM better captures sentence-level context but can propagate that context too aggressively.

Would I Have Switched to Neural in 2014?

Based on this experiment alone, I would not immediately replace the CRF when labeled data is very limited. At 5% and 10% of the training set, the CRF still performs better because its hand-designed features provide useful structure from few examples. However, the crossover occurs between 10% and 25%: at 10%, the CRF scores 0.8804 versus 0.8686 for the BiLSTM, while at 25% the BiLSTM leads 0.9271 to 0.9139. At full training size, the gap grows to 0.9686 versus 0.9522. Therefore, I would keep the CRF at very small data sizes but switch to the BiLSTM between **10% and 25%**, or about **448–1,120 labeled utterances**.

AI Use Disclosure

I used ChatGPT as an assistant throughout Assignment 1A. I used it to help generate and debug boilerplate code for the `train_loop()` and `viterbi_decode()` workflow, generate plotting code for the Part C data-efficiency curve, and create a script for identifying and organizing CRF-versus-BiLSTM test-set failures for Part D. I also used ChatGPT to clarify assignment instructions, troubleshoot PACE-ICE and GPU execution, check experimental procedures, and improve the organization and wording of my report. I independently ran all experiments on PACE-ICE, inspected the resulting predictions and evaluation scores, verified the final checkpoints, and reviewed the selected Part D failures. I used the generated error-analysis candidates as evidence for my own diagnoses rather than treating generated explanations as model outputs. I take responsibility for the submitted code, experimental results, interpretations, and report.

Appendix — Error Analysis Summary

Ten failures from the final CRF and BiLSTM models were examined: **four CRF-only errors, three BiLSTM-only errors, and three cases where both models were wrong.**

Example	Case	Main Error
1	CRF wrong	“New York City” and “next Saturday” incorrectly labeled as departure rather than arrival information.
2	CRF wrong	“Monday before 3 pm” recognized correctly as a span but assigned departure instead of arrival labels.
3	CRF wrong	“Wednesday before 5 pm” similarly mislabeled as departure information despite the cue “arrive.”
4	CRF wrong	“After 5 pm” assigned departure labels, and “soon” incorrectly tagged as a destination.
5	BiLSTM wrong	Date/time following “Las Vegas” incorrectly associated with the earlier cue “arrive.”
6	BiLSTM wrong	“Saturday April ninth” incorrectly labeled as an arrival date rather than departure date.
7	BiLSTM wrong	Correct return-date span found, but <code>date_relative</code> was confused with <code>today_relative</code> .
8	Both wrong	Ground-transportation locations received flight-oriented <code>fromloc / toloc</code> labels instead of generic <code>airport_name / city_name</code> .

Example	Case	Main Error
9	Both wrong	Both used route-specific labels for “downtown New York City”; the BiLSTM also confused “New York’s” with the airport span.
10	Both wrong	Both applied route/time-specific labels in a ground-transportation query where the gold labels were more generic.

Across these examples, the **CRF mainly confused arrival and departure roles while often preserving correct slot boundaries**. The **BiLSTM better captured broader context but sometimes propagated contextual cues too far or confused closely related slot categories**. Both models showed difficulty when ground-transportation queries required generic location labels rather than the route-oriented labels common in flight queries.